

Portfolio Watcher

Handover document (openstaande punten en advies voor vervolg)

Datum: 16 januari 2026 | **Doelgroep:** volgende ontwikkelaar(s) / beoordelaar / toekomstig team

1. Huidige status (samenvatting)

De basisfunctionaliteit voor portfolios en trades is aanwezig inclusief een 3-laagse architectuur, DTO-mapping, en een eerste opzet van DI. Er zijn echter nog verbeterpunten op het gebied van foutafhandeling, domeinmodellering (ownership trades), database-connection beheer, en verdere professionalisering (tests, logging, configuratie).

2. Must-do (hoogste prioriteit)

2.1 Domeinmodellering: Portfolio beheert Trades

- Refactor zodat Portfolio een collectie Trades beheert (en niet andersom).
- Zorg dat services/repositories hierop aansluiten (bijv. PortfolioService haalt trades erbij of laadt trades lazy).
- Werk class diagram + ERD + onderbouwing bij als bewijs voor LO1/LO2.

2.2 Centrale exception handling en user-friendly foutmeldingen

- Introduceer custom exception types per laag (DomainException, DataAccessException, ValidationException).
- Voeg ASP.NET middleware/filters toe voor globale exception handling: log systeemfouten, toon generieke foutpagina, voorkom info-leaks.
- Zorg dat user-correctable fouten als validatie/conflictmeldingen terugkomen in de UI i.p.v. 500-pages.

2.3 Connection management (Connection Factory / per request)

- Maak een ConnectionFactory die per operatie een SqlConnection opent en weer sluit (using).
- Vermijd een globale connection die altijd open staat (concurrency-issues, resource-leaks).
- Zet connection string in configuratie (appsettings) en injecteer via options/constructor i.p.v. hardcoded of static.

3. Should-do (kwaliteit en onderhoudbaarheid)

3.1 Geldtypes: decimal i.p.v. double/float

Voor geld is decimal in .NET doorgaans veiliger (minder afrondingsfouten). Overweeg migratie van BuyPrice/SellPrice naar decimal in domein, DTOs en database. Voeg tests toe voor afronding en berekeningen.

3.2 DI en composition root opschonen

- Zorg dat PageModels geen new() meer doen voor repositories/services.
- Registreer interfaces en implementaties in Program.cs (Scoped waar passend).
- Overweeg interfaces voor services als mocking nodig is.

3.3 Mapping en viewmodels

- Evalueer of je overal Trade -> TradeView mapping nodig hebt; vermijd dubbele mapping waar het geen waarde toevoegt.
- Als je ViewModels houdt: map in een aparte mapper class om SRP te houden (geen mapping in PageModels).

3.4 Unit tests uitbreiden (LO4)

- Schrijf tests voor Trade-berekeningen (happy + sad flow).
- Test validatie in constructors/methodes (bijv. negatieve shares, sellprice < 0, buyprice <= 0).
- Gebruik fake/mock repositories zodat tests niet afhankelijk zijn van de database.
- Test dat juiste custom exceptions gegoooid worden in sad flows.

3.5 Logging en observability

- Gebruik ILogger<T> voor logging in middleware en data-laag.
- Log database fouten met correlation id, maar toon de gebruiker alleen een generieke melding.

4. Could-do (uitbreidingen / scope optioneel)

4.1 Symbol prijsdata: van mock naar echte bron

- Vervang random/mock current price door een echte API of door een aparte prijs-tabel.
- Houd symbol prijslogica uit het domein; laat een repository de data leveren zodat het in tests gemockt kan worden.

4.2 Prijshistorie en audit trail

- Als de scope dit toelaat: introduceer een price history tabel per symbol.
- Overweeg audit trail voor wijzigingen aan trades (wie/wanneer/wat), alleen als dit echt requirement wordt.

4.3 Database constraints en integriteit

- Overweeg unieke constraints (bijv. portfolio naam per user) en handel conflicts netjes af in de UI.
- Zorg voor duidelijke foreign keys en indexes op veelgebruikte queries.

5. Bekende valkuilen / lessons learned

- Let op locale/culture issues bij decimal input (punt vs komma) en test dit expliciet.
- Trade-ownership: voorkom dubbele relaties die hetzelfde afdwingen (db + domein).

- Gebruik geen globale open database-connection; schaalbaarheid en stabiliteit gaan snel stuk.